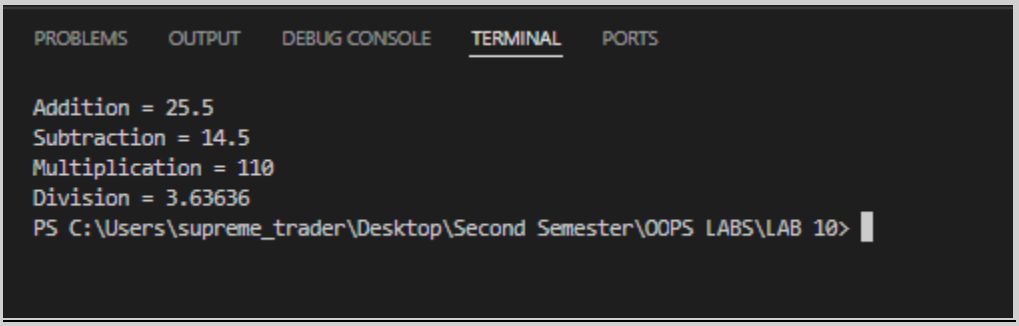
QUESTION NO#01:

```
#include <iostream>
using namespace std;

template <class T1, class T2>
class Calculator {
private:
  T1 num1;
  T2 num2;

public:
  Calculator(T1 a, T2 b) {
    num1 = a;
    num2 = b;
  }

  void add() {
    cout << "Addition = " << num1 + num2 << endl;
  }

  void subtract() {
    cout << "Subtraction = " << num1 - num2 << endl;
  }

  void multiply() {
```

```
        cout << "Multiplication = " << num1 * num2 << endl;
    }

    void divide() {
        if (num2 != 0) {
            cout << "Division = " << num1 / num2 << endl;
        } else {
            cout << "Division by zero is not possible" << endl;
        }
    }
};

int main() {
    Calculator<int, double> c1(20, 5.5);

    c1.add();
    c1.subtract();
    c1.multiply();
    c1.divide();

    return 0;
}
```

OUTPUT:

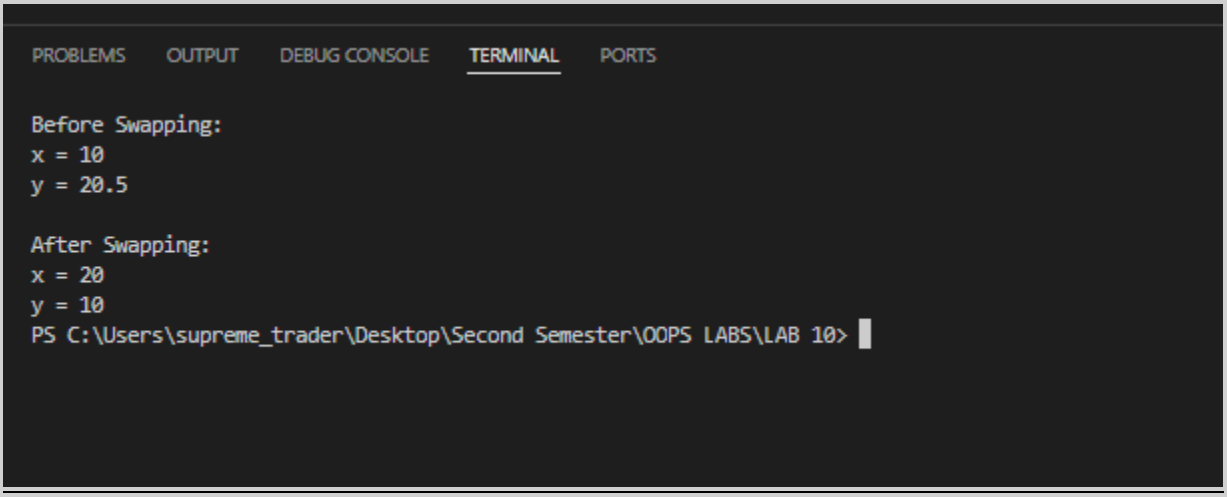


QUESTION NO#02:

```
#include <iostream>

using namespace std;


template <class T1, class T2>

void swapData(T1 &a, T2 &b) {

   T1 temp = a;

   a = b;

   b = temp;

}


int main() {

   int x = 10;

   double y = 20.5;


   cout << "Before Swapping:" << endl;

   cout << "x = " << x << endl;

   cout << "y = " << y << endl;
```

```
    swapData(x, y);


    cout << "\nAfter Swapping:" << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;


    return 0;
}
```

OUTPUT:



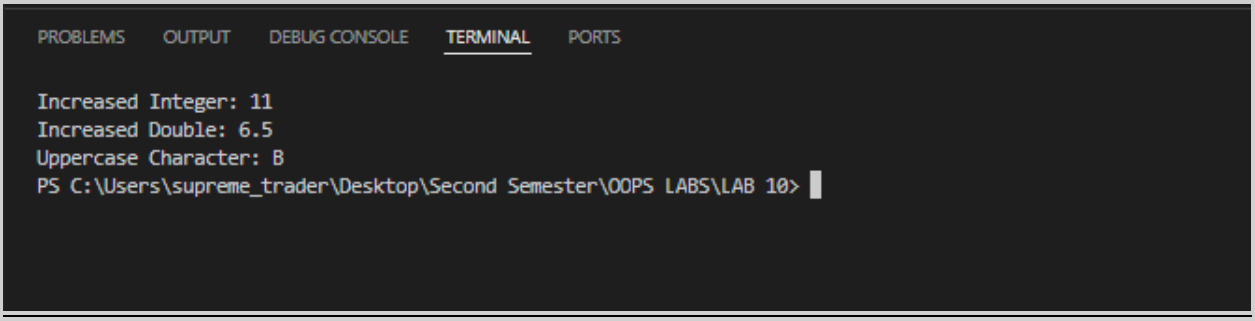
QUESTION NO#03:

```
#include <iostream>
using namespace std;


template <class T>
class mycontainer {
private:
    T element;
```

```
public:
   mycontainer(T arg) {
      element = arg;
   }

   T increase() {
      return ++element;
   }
};

template <>
class mycontainer<char> {
private:
   char element;

public:
   mycontainer(char arg) {
      element = arg;
   }

   char uppercase() {
      if (element >= 'a' && element <= 'z') {
         element = element - 32;
      }
      return element;
```

```
    }
};

int main() {
    mycontainer<int> obj1(10);
    cout << "Increased Integer: " << obj1.increase() << endl;

    mycontainer<double> obj2(5.5);
    cout << "Increased Double: " << obj2.increase() << endl;

    mycontainer<char> obj3('b');
    cout << "Uppercase Character: " << obj3.uppercase() << endl;

    return 0;
}
```

OUTPUT:



QUESTION NO#04:

```
#include <iostream>
using namespace std;

template <class T>
class DynamicArray {
protected:
   T* arr;
   int capacity;

public:
   DynamicArray(int size = 5) {
      capacity = size;
      arr = new T[capacity];
   }

   virtual bool isFull() = 0;
   virtual bool isEmpty() = 0;
   virtual int size() = 0;
   virtual T Front() = 0;
   virtual T Rear() = 0;
   virtual void enqueue(T value) = 0;
   virtual void dequeue() = 0;
   virtual void resize() = 0;
```

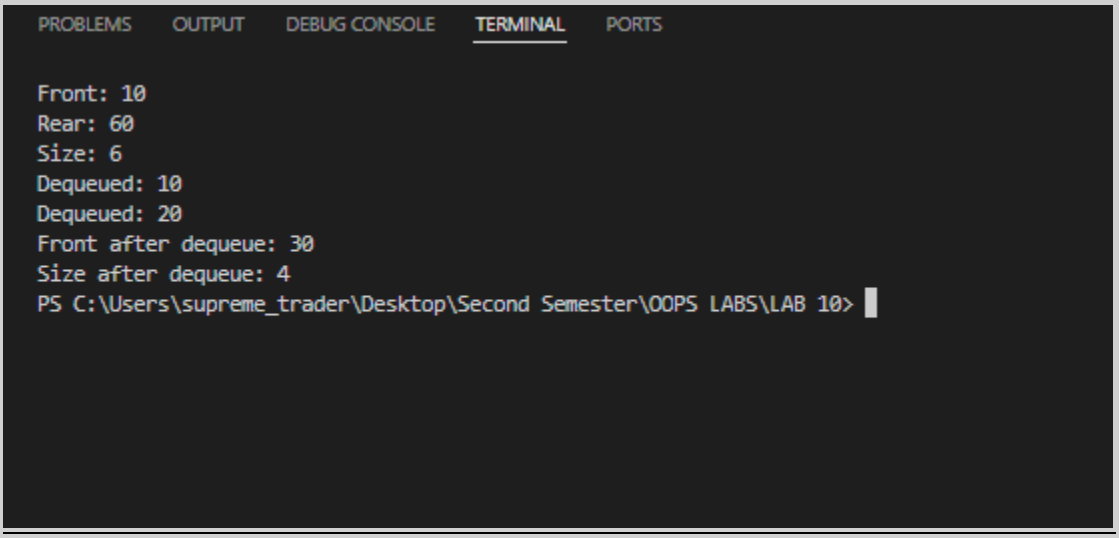
```
    virtual ~DynamicArray() {
        delete[] arr;
    }
};

template <class T>
class Queue : public DynamicArray<T> {
private:
    int front;
    int rear;
    int count;

public:
    Queue(int size = 5) : DynamicArray<T>(size) {
        front = 0;
        rear = -1;
        count = 0;
    }

    bool isFull() {
        return count == this->capacity;
    }

    bool isEmpty() {
        return count == 0;
    }
```

```
int size() {
    return count;
}

T Front() {
    return this->arr[front];
}

T Rear() {
    return this->arr[rear];
}

void enqueue(T value) {
    if (isFull()) {
        resize();
    }

    rear = (rear + 1) % this->capacity;
    this->arr[rear] = value;
    count++;
}

void dequeue() {
    if (isEmpty()) {
        cout << "Queue is Empty" << endl;
```

```
        return;
    }

    cout << "Dequeued: " << this->arr[front] << endl;
    front = (front + 1) % this->capacity;
    count--;
  }

  void resize() {
    int newCapacity = this->capacity * 2;
    T* temp = new T[newCapacity];

    for (int i = 0; i < count; i++) {
      temp[i] = this->arr[(front + i) % this->capacity];
    }

    delete[] this->arr;
    this->arr = temp;

    front = 0;
    rear = count - 1;
    this->capacity = newCapacity;
  }
};

int main() {
```

```
    Queue<int> q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60);

    cout << "Front: " << q.Front() << endl;
    cout << "Rear: " << q.Rear() << endl;
    cout << "Size: " << q.size() << endl;

    q.dequeue();
    q.dequeue();

    cout << "Front after dequeue: " << q.Front() << endl;
    cout << "Size after dequeue: " << q.size() << endl;

    return 0;
}
```

OUTPUT:



QUESTION NO#05:

```
#include <iostream>
#include <string>
using namespace std;

template <class T>
class DynamicArray {
protected:
   T* arr;
   int capacity;

public:
   DynamicArray(int size = 5) {
      capacity = size;
      arr = new T[capacity];
   }
```

```
    virtual bool isFull() = 0;
    virtual bool isEmpty() = 0;
    virtual int size() = 0;
    virtual T Front() = 0;
    virtual T Rear() = 0;
    virtual void enqueue(T value) = 0;
    virtual void dequeue() = 0;
    virtual void resize() = 0;

    virtual ~DynamicArray() {
        delete[] arr;
    }
};

template <class T>
class Queue : public DynamicArray<T> {
private:
    int front;
    int rear;
    int count;

public:
    Queue(int size = 5) : DynamicArray<T>(size) {
        front = 0;
        rear = -1;
```

```
    count = 0;
}

bool isFull() {
    return count == this->capacity;
}

bool isEmpty() {
    return count == 0;
}

int size() {
    return count;
}

T Front() {
    return this->arr[front];
}

T Rear() {
    return this->arr[rear];
}

void enqueue(T value) {
    if (isFull()) {
        resize();
```

```
    }

    rear = (rear + 1) % this->capacity;
    this->arr[rear] = value;
    count++;

    cout << value << " added to print queue" << endl;
}

void dequeue() {
    if (isEmpty()) {
        cout << "No print jobs in queue" << endl;
        return;
    }

    cout << "Printing Job: " << this->arr[front] << endl;

    front = (front + 1) % this->capacity;
    count--;
}

void resize() {
    int newCapacity = this->capacity * 2;
    T* temp = new T[newCapacity];

    for (int i = 0; i < count; i++) {
```

```
        temp[i] = this->arr[(front + i) % this->capacity];
    }

    delete[] this->arr;
    this->arr = temp;

    front = 0;
    rear = count - 1;
    this->capacity = newCapacity;
  }
};

int main() {
  Queue<string> printerQueue;

  printerQueue.enqueue("Assignment.pdf");
  printerQueue.enqueue("ProjectReport.docx");
  printerQueue.enqueue("CV.pdf");
  printerQueue.enqueue("Presentation.pptx");

  cout << endl;

  while (!printerQueue.isEmpty()) {
    printerQueue.dequeue();
  }
```

```
    cout << endl;
    cout << "All print jobs completed" << endl;


    return 0;
}
```

OUTPUT: